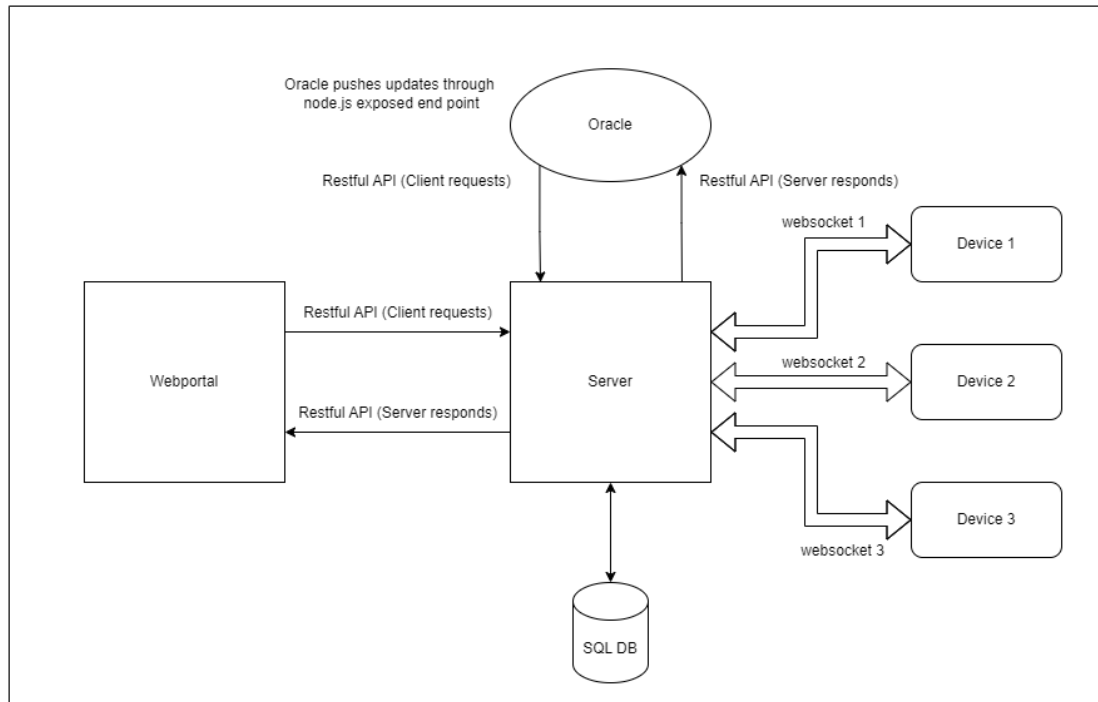


AI-CARING Architecture



Architectural Diagram Components

Node.js Server

Purpose: Acts as the central hub for managing connections and data flow between the client devices (e.g., iPads) and the MySQL database. It handles logic for sending reminders, processing user inputs, and interacting with external services for utility status checks. Additionally, it integrates with Firebase for user authentication.

Interactions:

- Exposes an API endpoint to receive real-time updates on utility status from Oracle.
- Communicates with client devices using WebSocket for real-time data exchange.
- Performs CRUD operations on the MySQL database.
- Handles user authentication requests using JWT.

Technology Used: Node.js for the server framework, WebSocket for real-time communication.

MySQL Database

Purpose: Stores and manages structured data, including user profiles and reminders. MySQL is chosen for its robustness, reliability, and support for complex queries and transactions, ensuring data integrity.

Interactions: Receives queries from the Node.js server for storing, updating, retrieving, and deleting user profiles and reminders. Supports transactions to ensure data consistency.

Technology Used: MySQL as the database management system.

AI-CARING Architecture

Client Devices (iPads)

Purpose: Provides the interface for users to interact with the system, displaying reminders and allowing users to manage their profiles and reminder settings.

Interactions: Sends user inputs to the Node.js server and receives real-time reminders and updates via WebSocket connection. Authenticates users through Firebase.

Technology Used: iOS SDK for developing the client application.

Web Portal

Purpose: Offers a web-based portal for users to create, update, and manage reminders and their profiles. Utilizes an alternative authentication system for user authentication, leveraging support for secure session management.

Interactions: Communicates with the Node.js server to perform CRUD operations on the MySQL database. Handles user sign-in and sign-up processes through the alternative authentication system.

Technology Used: HTML, CSS, JavaScript Frameworks for the web interface

Oracle (Utility Status)

Purpose: Provides real-time data on the status of kitchen utilities (e.g., whether the fridge is closed), enhancing the context-awareness of the reminders.

Interactions:

- Pushes updates to the Node.js server's API endpoint when there is activity with the utilities.

Technology Used: NA.

User Action and System Response Process

User Creates a Reminder: The user accesses the web portal and creates a reminder. The reminder could be based on specific conditions like "EVERYTIME after dinner SHOW..." or "IF 7pm SHOW...". The time parameter is optional, and the reminder could be triggered by various events or conditions.

Reminder Storage: Once created, the reminder details are sent to the Node.js server, which then stores the reminder in the MySQL database. The reminder includes the condition for triggering it, such as a specific time, utility status change, or user activity.

Oracle Pushes Updates: Oracle monitors the status of various utilities and user activities. Whenever there is a change that matches the conditions set in any of the reminders (e.g., "EVERYTIME Microwave door is open SHOW..."), Oracle pushes these updates to the Node.js server.

Node.js Server Processes Updates: Upon receiving updates from Oracle, the Node.js server checks the MySQL database for any reminders created for the specific event or condition reported in the update.

Sending the Reminder: If a matching reminder is found, the Node.js server sends the reminder to the client device (e.g., an iPad) using WebSocket for real-time communication. This ensures that reminders are delivered promptly and efficiently.

AI-CARING Architecture

Displaying the Reminder: The client device receives the reminder and displays it to the user. This could be in the form of a notification, a message on the web portal, or any other specified action.

Example Scenarios

- "EVERYTIME after dinner SHOW...": The system could use a combination of time and activity sensors to determine "after dinner" times. Once this condition is met, a reminder is sent to the user's device.
- "IF 7pm SHOW...": The system checks the current time, and when it's 7pm, it triggers the reminder.
- "EVERYTIME Microwave door is open SHOW...": Oracle sends an update to the Node.js server every time the microwave door is opened. The server then checks for any reminders related to this event and sends them to the user's device.
- "NEXT TIME 7pm on Wednesday SHOW...": The system would need to track both the day of the week and the time to trigger this reminder the next time these conditions are met.
- "EVERYTIME person is cooking AND today's recipe is pasta SHOW": This scenario might involve more complex sensors or inputs, including recipe recognition. Once both conditions are satisfied, the reminder is triggered.

This architecture leverages real-time data and user-defined conditions to provide timely and relevant reminders, enhancing the user experience by making the reminders context-aware and personalized. The use of WebSocket ensures that the communication between the server and client devices is efficient, allowing for immediate notification delivery.

System Process for the Trigger "EVERYTIME after dinner SHOW..."

- **User Sets a Reminder:** The user logs into the web portal and sets a new reminder with the condition "EVERYTIME after dinner SHOW...". The user specifies the action to be taken by the system when the condition is met.
- **Reminder Submission:** The reminder details are submitted to the Node.js server via an HTTP request.
- **Reminder Storage:** The Node.js server processes the request and stores the reminder in the MySQL database, associating it with the user's profile and the specified condition.
- **Dinner Activity Detection:** The system needs a way to detect the "after dinner" condition. This could involve user input, a scheduled time, or sensors that detect activity in the dining area.
- **Oracle Utility Status Update:** If the system uses sensors to detect the "after dinner" condition, Oracle would push updates to the Node.js server when it detects activity indicating dinner has concluded (e.g., the dishwasher starts, or the kitchen lights turn off).
- **Condition Verification:** The Node.js server continuously listens for updates from Oracle. When it receives an update that matches the "after dinner" condition, it checks the database for any reminders set for "after dinner".
- **Triggering the Reminder:** If a matching reminder is found, the Node.js server triggers the reminder.
- **Sending the Reminder:** The server sends the reminder to the user's device via WebSocket, ensuring real-time delivery.

AI-CARING Architecture

- **Displaying the Reminder:** The client device (iPad) receives the reminder and displays a notification or alert to the user, prompting them to take the specified action.
- **User Acknowledges:** The user sees the reminder and can then acknowledge it, complete the action, or dismiss it.

Descriptions of the format of the inputs from the Oracle.

JSON Structure

The JSON input consists of an array of objects, each representing a home. Each home object contains a unique home identifier and an array of utilities. Each utility within the array has a unique utility identifier, a name, and a status.

```
{
  "home_utilities": [
    {
      "home_id": "string",
      "utilities": [
        {
          "utility_id": "string",
          "utility_name": "string",
          "status": "string",
          "components": [
            {
              "component_name": "string",
              "status": "string"
            }
            // Additional components can be added as needed
          ]
        }
      ],
      "activities": [
        {
          "activity_id": "string",
          "activity_type": "string",
          "status": "string"
        }
        // Additional activities can be added as needed
      ]
    }
  ]
}
```

Fields Description:

- **home_utilities:** Root element; an array of home objects.
- **home_id:** Unique identifier for the home.
- **utilities:** Array of utility objects within the home.

AI-CARING Architecture

- **utility_id**: Unique identifier for the utility.
- **utility_name**: Name of the utility (e.g., Microwave, Fridge).
- **status**: Current status of the utility (e.g., On, Off).
- **components**: An array of objects, each representing a component of the utility with its status (e.g., door: "Open").
- **activities**: An array of activity objects representing various activities within the home.
 - **activity_id**: Unique identifier for the activity.
 - **activity_type**: Type of activity (e.g., Cooking, Sleeping).
 - **status**: Current status of the activity (e.g., Active, Inactive).

Example

```
{
  "home_utilities": [
    {
      "home_id": "home_1296971",
      "utilities": [
        {
          "utility_id": "utility_microwave_001",
          "utility_name": "Microwave",
          "status": "On",
          "components": [
            {
              "component_name": "door",
              "status": "Closed"
            },
            {
              "component_name": "light",
              "status": "Off"
            }
          ]
        },
        {
          "utility_id": "utility_fridge_001",
          "utility_name": "Fridge",
          "status": "On",
          "components": [
            {
              "component_name": "door",
              "status": "Open"
            },
            {
              "component_name": "light",
              "status": "On"
            }
          ]
        }
      ]
    }
  ]
}
```

AI-CARING Architecture

```
    ]
  }
],
"activities": [
  {
    "activity_id": "activity_cooking_001",
    "activity_type": "Cooking",
    "status": "Active"
  },
  {
    "activity_id": "activity_sleeping_001",
    "activity_type": "Sleeping",
    "status": "Inactive"
  }
]
}
]
```

Usage Notes

- This JSON format is designed to be highly flexible, allowing for the addition of new utilities, components, and activities without altering the existing structure. It ensures that the system can adapt to various scenarios and activities within the home, providing a comprehensive overview of the home's status.
- Ensure that the JSON input conforms to RFC 8259 standards for well-formed JSON data